Module 18 - Web Scraping Advanced

This notebook covers **production-quality scraping**: handling authentication, pagination, retries, dynamic content, and the legal boundaries every professional must understand.

Contents

- [ ] 1. Headers and Sessions — Mimicking a Real Browser
- [ ] 2. Handling Pagination — Looping Across Multiple Pages
- [ ] 3. Robust Error Handling — Timeouts, Retries, Status Checks
- [ ] 4. CSS Selectors — Advanced Patterns
- [ ] 5. Dynamic Content — When JavaScript Renders the Page
- [ ] 6. Ethical and Legal Boundaries

---

1. Headers and Sessions — Mimicking a Real Browser

Why servers block raw `requests` calls

A raw `requests.get()` sends a minimal HTTP request with a default `User-Agent` header that looks like `"python-requests/2.31.0"`. Many servers detect this and return 403 Forbidden, a CAPTCHA, or empty content.

The solution is to send headers that look like a real browser.

The `User-Agent` header

The `User-Agent` is how a client identifies itself to a server. You can set any headers you want using the `headers=` argument:

```
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 ...",
    "Accept": "text/html,application/xhtml+xml,*/*",
    "Accept-Language": "en-US,en;q=0.9",
}
response = requests.get(url, headers=headers, timeout=10)
```

Warning — Headers are not a bypass

Setting a browser User-Agent to circumvent anti-bot measures may violate the site's Terms of Service. Use it to identify yourself honestly where needed, not to deceive security systems.

In [ ]:
```
import requests
from bs4 import BeautifulSoup
import time, json, csv, re

# Compare: default headers vs browser-like headers
# httpbin.org/headers echoes back the headers your request sent

print("=== Default python-requests headers ===")
default_response = requests.get("https://httpbin.org/headers", timeout=10)
default_headers = default_response.json().get("headers", {})
print(f"User-Agent: {default_headers.get('User-Agent', 'not sent')}")

print()

# Browser-like headers — reduces chance of being blocked on public data sites
browser_headers = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.5",
    "Connection": "keep-alive",
}

print("=== Browser-like headers ===")
browser_response = requests.get("https://httpbin.org/headers", headers=browser_head
browser_echoed = browser_response.json().get("headers", {})
print(f"User-Agent: {browser_echoed.get('User-Agent', 'not sent')}")
```

Sessions — Maintaining State Across Requests

A `requests.Session` object:

- **Persists cookies** automatically across all requests in the session
- **Reuses the TCP connection** — faster, and looks more like a real browser
- Lets you set headers once, instead of on every request

Use a Session whenever you are making multiple requests to the same server.

In [ ]:
```
# Using a Session — headers set once, applied to all requests
session = requests.Session()

session.headers.update({
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
```

```
    "Accept-Language": "en-US,en;q=0.5",
})

# All requests through this session share the headers and cookies
response_1 = session.get("https://httpbin.org/headers", timeout=10)
response_2 = session.get("https://httpbin.org/headers", timeout=10)

print("Both requests used the same session headers.")
print(f"r1 User-Agent: {response_1.json()['headers'].get('User-Agent', '')[:40]}...
print(f"r2 User-Agent: {response_2.json()['headers'].get('User-Agent', '')[:40]}...

session.close()
```

In [ ]:

```
# Session as context manager — automatically closes the connection
with requests.Session() as session:
    session.headers.update({"User-Agent": "SecurityResearchBot/1.0 (research@exampl

    # Cookies set on one request are sent on subsequent ones automatically
    r1 = session.get("https://httpbin.org/cookies/set/session_id/abc123", timeout=1
    r2 = session.get("https://httpbin.org/cookies", timeout=10)

    cookies = r2.json().get("cookies", {})
    print(f"Session cookie persisted: {cookies}")
```

Practice — Headers and Sessions

Exercise 1 — Write: Build a scraping session for a fictional threat intel portal.

1. Create a `requests.Session()`
2. Set a descriptive `User-Agent` that identifies your tool as a security research script
3. Make two requests to `https://httpbin.org/headers` using the session
4. From the second response, extract and print the `User-Agent` that was sent
5. Close the session properly (use a `with` block or `session.close()`)

In [ ]:

```
# Your code here
```

▶ 💡 **SOLUTION**

---

2. Handling Pagination — Looping Across Multiple Pages

Most real data sources split results across multiple pages. There are two common pagination patterns:

Pattern A — Predictable URL structure

The page number appears directly in the URL:

```
https://vulndb.example.com/cves?page=1
https://vulndb.example.com/cves?page=2
https://vulndb.example.com/cves?page=3
```

You loop from page 1 until you hit an empty page or a maximum.

In [ ]:
```python
# Pattern A: URL-based pagination
# We simulate multi-page data with mock HTML since we can't hit a real vuln DB

def make_mock_page(page_num, total_pages=3):
    """Generate a mock vulnerability listing page."""
    entries = {
        1: [("CVE-2024-1001", "CRITICAL", "9.8"), ("CVE-2024-1002", "HIGH", "7.5")]
        2: [("CVE-2024-1003", "HIGH", "7.1"), ("CVE-2024-1004", "MEDIUM", "5.4")],
        3: [("CVE-2024-1005", "MEDIUM", "4.0")],
    }
    items = entries.get(page_num, [])
    if not items:
        return "<html><body><div class='results'></div></body></html>"

    rows = ""
    for cve_id, sev, score in items:
        rows += f'<div class="vuln-row"><span class="cve">{cve_id}</span>'
        rows += f'<span class="sev">{sev}</span><span class="score">{score}</span><

    next_link = ""
    if page_num < total_pages:
        next_link = f'<a class="next-page" href="?page={page_num + 1}">Next</a>'

    return f"<html><body><div class='results'>{rows}</div>{next_link}</body></html>


# Simulate paginated scraping
all_vulns = []
max_pages = 5   # safety ceiling — never loop without a limit

for page in range(1, max_pages + 1):
    print(f"Scraping page {page}...")
    html = make_mock_page(page)              # in real code: requests.get(url + f"?
    soup = BeautifulSoup(html, "html.parser")

    rows = soup.select("div.vuln-row")
    if not rows:
        print(f"  Page {page} is empty — stopping.")
        break

    for row in rows:
        all_vulns.append({
            "cve_id": row.select_one(".cve").get_text(strip=True),
            "severity": row.select_one(".sev").get_text(strip=True),
            "score": float(row.select_one(".score").get_text(strip=True)),
        })
    print(f"  Found {len(rows)} entries on page {page}")
    time.sleep(0.5)   # polite delay between pages
```

```
print(f"\nTotal collected: {len(all_vulns)} vulnerabilities across {page} pages")
for v in all_vulns:
    print(f"  {v['cve_id']} | {v['severity']} | CVSS {v['score']}")
```

Pattern B — Following "Next" Links

Some sites don't use predictable URL structures — instead, each page contains a `<a>` link pointing to the next page. You follow that link until it disappears.

In [ ]:
```
# Pattern B: follow the "Next" link
# We reuse the mock page generator from above

all_vulns_b = []
current_page = 1

while True:
    html = make_mock_page(current_page)
    soup = BeautifulSoup(html, "html.parser")

    # Extract data from current page
    for row in soup.select("div.vuln-row"):
        all_vulns_b.append(row.select_one(".cve").get_text(strip=True))

    # Look for the "Next" link
    next_link = soup.select_one("a.next-page")
    if next_link is None:
        print(f"No next-page link on page {current_page} — done.")
        break

    # In real scraping: next_url = base_url + next_link.get("href")
    # Here we just increment the page counter
    current_page += 1
    time.sleep(0.5)

print(f"Collected {len(all_vulns_b)} CVEs via next-link pattern: {all_vulns_b}")
```

Practice — Pagination

Exercise 2 — Write: The function `make_mock_threat_feed(page)` below returns a mock HTML page of threat IPs. Pages 1–4 have data. Page 5 is empty.

Write a paginated scraper that:

1. Loops through pages using the URL pattern (Pattern A)
2. Stops when a page returns no entries
3. Collects all IPs into a list called `threat_ips_paginated`
4. Adds a 0.5-second delay between pages
5. Prints the total count at the end

In [ ]:
```
def make_mock_threat_feed(page):
    data = {
        1: ["45.33.32.156", "192.241.187.88"],
        2: ["185.220.101.5", "91.108.4.200"],
        3: ["103.251.167.20", "37.49.226.48"],
        4: ["5.188.206.14"],
    }
    ips = data.get(page, [])
    rows = "".join(f'<div class="ip-entry">{ip}</div>' for ip in ips)
    return f"<html><body><div class='feed'>{rows}</div></body></html>"

# Your scraping loop here
```

▶ 💡 **SOLUTION**

---

3. Robust Error Handling — Timeouts, Retries, Status Checks

A scraper that crashes on the first error is not production-ready. Networks are unreliable. Servers go down. Rate limits kick in. Robust scrapers **handle failures gracefully and retry intelligently**.

The full exception hierarchy for `requests`

```
requests.exceptions.RequestException       <- parent of all
    ConnectionError                         <- no route to host,
DNS failure
    Timeout                                 <- response too slow
    TooManyRedirects                        <- redirect loop
    HTTPError                               <- raised by
raise_for_status()
```

Retry pattern with exponential backoff

Exponential backoff means: wait longer after each consecutive failure. If the server is struggling, hammering it faster makes things worse for everyone.

In [ ]:
```
import time

def fetch_with_retry(url, session, max_retries=3, base_delay=2):
    """
    Fetch a URL with automatic retries and exponential backoff.

    Parameters
    ----------
    url        : str   — target URL
    session    : requests.Session
```

```
    max_retries: int   — how many times to retry on failure
    base_delay : float — initial wait in seconds (doubles each retry)

    Returns
    -------
    requests.Response or None
    """
    for attempt in range(1, max_retries + 1):
        try:
            response = session.get(url, timeout=10)
            response.raise_for_status()
            return response                        # success — return immediately

        except requests.exceptions.HTTPError as e:
            status = e.response.status_code if e.response else "unknown"
            if status == 429:
                wait = base_delay * (2 ** (attempt - 1))    # 2, 4, 8 seconds
                print(f"  429 Too Many Requests — waiting {wait}s before retry {att
                time.sleep(wait)
            elif status in (500, 502, 503):
                wait = base_delay * attempt
                print(f"  Server error {status} — waiting {wait}s before retry {att
                time.sleep(wait)
            else:
                print(f"  HTTP {status} — not retrying (client error)")
                return None

        except requests.exceptions.Timeout:
            print(f"  Timeout on attempt {attempt}/{max_retries}")
            time.sleep(base_delay * attempt)

        except requests.exceptions.ConnectionError:
            print(f"  Connection error on attempt {attempt}/{max_retries}")
            time.sleep(base_delay * attempt)

    print(f"  All {max_retries} attempts failed for {url}")
    return None


# Test the function
with requests.Session() as session:
    session.headers.update({"User-Agent": "SecurityResearchBot/1.0"})

    result = fetch_with_retry("https://httpbin.org/get", session)
    if result:
        print(f"Fetch succeeded: status {result.status_code}")
    else:
        print("Fetch failed after all retries")
```

Defensive extraction — guard every `.find()` result

In real pages, elements may be missing, have different class names, or be structured differently than you expected. Always guard against `None`.

Pattern to always follow:

```
element = soup.find("span", class_="score")
score = float(element.get_text(strip=True)) if element else None
```

In [ ]:
```
# Mock HTML with one incomplete record — missing the score field
incomplete_html = """
<html><body>
  <div class="vuln-entry">
    <h2 class="cve-id">CVE-2024-0001</h2>
    <span class="severity">CRITICAL</span>
    <span class="score">9.8</span>
  </div>
  <div class="vuln-entry">
    <h2 class="cve-id">CVE-2024-0002</h2>
    <span class="severity">HIGH</span>
    <!-- score tag is missing in this record -->
  </div>
</body></html>
"""

soup_incomplete = BeautifulSoup(incomplete_html, "html.parser")

results = []
for entry in soup_incomplete.find_all("div", class_="vuln-entry"):
    cve_tag      = entry.find("h2", class_="cve-id")
    severity_tag = entry.find("span", class_="severity")
    score_tag    = entry.find("span", class_="score")

    cve_id   = cve_tag.get_text(strip=True)      if cve_tag      else "UNKNOWN"
    severity = severity_tag.get_text(strip=True)  if severity_tag else "UNKNOWN"
    score    = float(score_tag.get_text(strip=True)) if score_tag else None

    results.append({"cve_id": cve_id, "severity": severity, "score": score})
    print(f"  Extracted: {cve_id} | {severity} | score={score}")

print(f"\nProcessed {len(results)} records (no crash on missing data)")
```

Practice — Robust Error Handling

Exercise 3 — Write: Write a function `safe_scrape(url, session)` that:

1. Fetches `url` using the session
2. Handles `Timeout`, `ConnectionError`, and `HTTPError` separately — printing a descriptive message for each
3. Returns the `BeautifulSoup` object on success, or `None` on any failure
4. Always adds a 1-second delay after the request (success or failure)

Test it against `https://httpbin.org/get` (should succeed) and `https://httpbin.org/status/503` (should fail gracefully).

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

4. CSS Selectors — Advanced Patterns

You already know `.classname`, `#id`, and `tag`. These are just the basics. CSS selectors can be extremely precise — useful when a page has many similarly-structured elements.

Selector reference

Selector	Example	Matches
`tag`	`div`	All `<div>` elements
`.class`	`.score`	Elements with class `score`
`#id`	`#cve-001`	Element with id `cve-001`
`tag.class`	`span.severity`	`<span>` with class `severity`
`parent child`	`div.vuln span`	`<span>` inside any `div.vuln`
`parent > child`	`div > span`	Direct child only
`[attr]`	`[href]`	Elements that have an `href` attribute
`[attr="val"]`	`[class="score"]`	Exact attribute value
`[attr*="val"]`	`[href*="cve"]`	Attribute contains "cve"
`[attr^="val"]`	`[href^="/vuln"]`	Attribute starts with "/vuln"
`:nth-child(n)`	`tr:nth-child(2)`	Second `<tr>`
`:not(.class)`	`:not(.resolved)`	Elements without that class

In [ ]:
```
mock_page = """
<html><body>
<div class="vuln-list">
  <div class="vuln-entry critical" id="cve-2024-0001">
    <span class="cve-id">CVE-2024-0001</span>
    <span class="score">9.8</span>
    <a href="/vuln/CVE-2024-0001">Details</a>
  </div>
  <div class="vuln-entry high resolved" id="cve-2024-0002">
    <span class="cve-id">CVE-2024-0002</span>
    <span class="score">7.5</span>
    <a href="/vuln/CVE-2024-0002">Details</a>
  </div>
  <div class="vuln-entry medium" id="cve-2024-0003">
    <span class="cve-id">CVE-2024-0003</span>
```

```
    <span class="score">4.3</span>
    <a href="/vuln/CVE-2024-0003">Details</a>
  </div>
</div>
</body></html>
"""

s = BeautifulSoup(mock_page, "html.parser")

# Attribute contains selector — links whose href contains "cve"
cve_links = s.select("a[href*='vuln']")
print("Links to vuln details:")
for link in cve_links:
    print(f"  {link.get_text(strip=True)} -> {link['href']}")
```

In [ ]:
```
# :not() selector — skip resolved vulnerabilities
# Entries that have class "vuln-entry" but NOT "resolved"
open_vulns = s.select("div.vuln-entry:not(.resolved)")

print(f"Open (unresolved) vulnerabilities: {len(open_vulns)}")
for entry in open_vulns:
    cve = entry.select_one(".cve-id").get_text(strip=True)
    print(f"  {cve}")
```

In [ ]:
```
# Descendant vs direct child
# All spans inside .vuln-list (any depth)
all_spans = s.select(".vuln-list span")
print(f"All spans anywhere in vuln-list: {len(all_spans)}")

# Direct child divs of .vuln-list only
direct_divs = s.select(".vuln-list > div")
print(f"Direct child divs of vuln-list: {len(direct_divs)}")
```

In [ ]:
```
# Attribute starts-with — useful for protocol or path filtering
# In a real page this might select only HTTPS links
https_links = s.select("a[href^='/vuln']")
print(f"Links starting with '/vuln': {[l['href'] for l in https_links]}")
```

Practice — CSS Selectors

Exercise 4 — Write: Using the `mock_page` and soup object `s` defined above:

1. Use a CSS selector to find all `<div>` elements that have **both** class `vuln-entry` and class `critical`
2. Use `:nth-child(2)` to select the second `.vuln-entry` inside `.vuln-list`
3. Use `[id^="cve-2024"]` to select all entries whose `id` attribute starts with `"cve-2024"`
4. Print the CVE ID from each result

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

5. Dynamic Content — When JavaScript Renders the Page

The problem

`requests` fetches only the **static HTML** the server sends. Many modern websites render their content using **JavaScript** that runs in the browser after the page loads. When you fetch these pages with `requests`, you get the empty shell — no data.

Signs that a page is dynamically rendered:

- `response.text` contains almost no content, just a `<div id="app"></div>` or similar
- The page source (Ctrl+U in browser) looks bare compared to what you see in the browser
- Browser DevTools shows XHR/fetch requests loading the actual data

Solution A — Find the API (best approach)

Most modern sites load their data from a **JSON API endpoint**. Check the browser's Network tab (F12 > Network > XHR/Fetch) while the page loads. You will often find a request like `https://api.example.com/vulnerabilities?page=1` that returns clean JSON — much easier to parse than HTML.

```
# Scraping the API directly — faster and more stable than scraping HTML
api_url = "https://api.example.com/vulnerabilities?page=1"
response = requests.get(api_url, headers=headers, timeout=10)
data = response.json()   # directly parsed — no BeautifulSoup needed
```

Solution B — Selenium (when you truly need a browser)

Selenium controls a real browser (Chrome, Firefox) from Python. The browser runs JavaScript, so you get the fully rendered page.

```
    pip install selenium
    pip install webdriver-manager   # handles browser driver
    installation

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Launch headless Chrome (no visible window)
```

```
options = webdriver.ChromeOptions()
options.add_argument("--headless")
options.add_argument("--no-sandbox")
driver = webdriver.Chrome(options=options)

driver.get("https://example.com/dynamic-page")

# Wait for a specific element to appear (do not use time.sleep blindly)
wait = WebDriverWait(driver, timeout=10)
element = wait.until(EC.presence_of_element_located((By.CLASS_NAME, "vuln-
entry")))

html = driver.page_source
soup = BeautifulSoup(html, "html.parser")
# ... extract data as normal

driver.quit()   # always close the browser
```

When to use which approach

Situation	Use
Static HTML page	`requests` + BeautifulSoup
Page loads data from a JSON API	`requests` against the API URL directly
JavaScript-rendered page, no API	Selenium
Page requires login and 2FA	Selenium (last resort)

Selenium is slower, heavier, and harder to maintain. **Always try `requests` first** — check the Network tab for API calls before resorting to Selenium.

In [ ]:
```
# Demonstration: detecting if a page requires JavaScript
# If response.text is very short or contains no data, it's likely dynamic

sample_response_text = """
<!DOCTYPE html>
<html>
  <head><title>Vulnerability Dashboard</title></head>
  <body>
    <div id="app"></div>
    <script src="/static/main.js"></script>
  </body>
</html>
"""

soup_check = BeautifulSoup(sample_response_text, "html.parser")
data_elements = soup_check.find_all(["div", "table", "li", "tr"])

print(f"Total data elements found: {len(data_elements)}")
print(f"Page is likely dynamic: {len(data_elements) <= 2}")
print()
```

```
print("Next step: open DevTools > Network > XHR/Fetch")
print("Look for API calls that return JSON — scrape those instead.")
```

---

6. Ethical and Legal Boundaries

This section is not optional. A cybersecurity professional who scrapes without understanding the legal context is a liability — to themselves and their employer.

`robots.txt` — good faith, not law

`robots.txt` is a convention. Python ignores it if you don't check. But courts in multiple jurisdictions have ruled that ignoring `robots.txt` after being notified can be relevant in computer access law cases. Always check it. Always respect it.

Terms of Service

Most websites have ToS that prohibit automated access, scraping, or bulk downloading. Violating ToS is a civil matter (not automatically criminal), but can result in:

- Account termination
- IP bans
- Cease-and-desist letters
- Civil lawsuits in extreme cases

Read the ToS before scraping a commercial site.

Computer access laws — varies by country

Law	Jurisdiction	Relevance
CFAA (Computer Fraud and Abuse Act)	USA	Unauthorized access — broad and aggressively interpreted
Computer Misuse Act 1990	UK	Unauthorized access to computer material
Belgian Computer Crime Law	Belgium / EU	Similar scope to CMA
GDPR	EU	Collecting personal data without legal basis — severe fines

Unauthorised is the key word. Scraping public data you are permitted to access is generally safe. Scraping data behind a login without permission, or data explicitly marked off-limits in `robots.txt`, enters legally grey or explicitly prohibited territory.

When scraping is clearly acceptable

- Public government databases (NVD, CISA KEV, Shodan public data)
- Open threat intelligence feeds (AbuseIPDB public API, VirusTotal public data)
- Certificate transparency logs (crt.sh)
- Your own infrastructure
- Sites that explicitly offer scraping-friendly APIs or Creative Commons data

Professional rules of thumb

1. **Prefer official APIs over scraping** — most security data sources have them
2. **Check `robots.txt` every time** — it changes
3. **Never collect personal data without a legal basis** — even if it's public
4. **Rate-limit aggressively** — 1–2 seconds minimum, honour `Crawl-delay`
5. **Identify your bot** — use a User-Agent that includes contact info
6. **When in doubt, ask** — email the site owner for permission

User-Agent best practice for professional bots

```
{
    "User-Agent": "SecurityResearchBot/1.0 (research; contact:
soc@yourcompany.com)"
}
```

This is honest, professional, and gives site admins a way to contact you if there is a problem.

In [ ]:
```
# Summary: a checklist function to run before any scraping project

def pre_scrape_checklist(base_url):
    """
    Run through the minimum professional checklist before scraping.
    Prints guidance — does not make decisions for you.
    """
    print(f"=== Pre-Scrape Checklist for {base_url} ===")
    print()

    print("[1] Check robots.txt")
    try:
        r = requests.get(base_url.rstrip("/") + "/robots.txt", timeout=10)
        if r.status_code == 200:
            disallowed = [l for l in r.text.split("\n") if "Disallow" in l]
            print(f"    Found robots.txt — {len(disallowed)} Disallow rule(s)")
            crawl_delay = [l for l in r.text.split("\n") if "Crawl-delay" in l]
            if crawl_delay:
                print(f"    {crawl_delay[0].strip()}")
        else:
            print(f"    No robots.txt (status {r.status_code}) — proceed carefully"
    except Exception as e:
        print(f"    Could not fetch robots.txt: {e}")

    print()
    print("[2] Manual checks needed (cannot automate these):")
    print("    - Read the site's Terms of Service")
```

```
    print("    - Confirm the data you need is public and not behind a login")
    print("    - Check whether an official API exists (prefer it over scraping)")
    print("    - Consider whether any data you collect could be personal data (GDPR
    print()
    print("[3] Technical hygiene:")
    print("    - Set a descriptive User-Agent with contact info")
    print("    - Use time.sleep() between requests (min 1-2 seconds)")
    print("    - Set timeout= on every request")
    print("    - Log errors — do not silently discard them")
    print()
    print("=== Proceed only after completing all checks ===")

pre_scrape_checklist("https://crt.sh")
```

Summary

Advanced scraping workflow

```
Pre-flight
  -> check robots.txt
  -> read Terms of Service
  -> look for an official API first

Setup
  -> requests.Session() with descriptive User-Agent
  -> retry function with exponential backoff

Per-page loop
  -> fetch_with_retry(url, session)
  -> BeautifulSoup(response.text, "html.parser")
  -> defensive extraction — guard every find() result with if-not-
None
  -> append to results list
  -> time.sleep(crawl_delay)
  -> follow pagination (URL pattern or next-link)

Post-collection
  -> save to CSV or JSON
  -> log summary: pages fetched, records collected, errors
encountered
```

Key concepts at a glance

Concept	What to remember
Session	Headers and cookies persist; reuse TCP connection
Retry + backoff	Wait 2x longer after each failure; cap at 3–5 retries
Defensive extraction	Every `find()` result can be `None` — always check

Concept	What to remember
Dynamic pages	Check Network tab for JSON APIs before using Selenium
`robots.txt`	Non-technical obligation — always check, always respect
Rate limiting	Minimum 1–2s between requests; honour `Crawl-delay`

Next Steps

02_Drills_Scraping.ipynb — 15 exercises across both ELI5 and Advanced topics